



CSCE 633-700: Machine Learning

Neural Networks (Part 1)

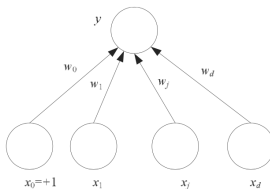
Overview (1)

- Perceptron
 - Representation
 - Learning
 - Examples
- Multilayer Perceptron
 - Representation
 - Learning: Backpropagation
 - Practical issues

Overview (2)

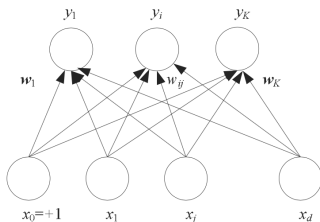
- Perceptron
 - Representation
 - Training
 - Examples
- Multilayer Perceptron
 - Representation
 - Learning: Backpropagation
 - Practical issues

Perceptron: Basic processing unit (1)



- Inputs $x_d \in \mathbb{R}$, $d = 1, \dots, D$
 - might come from the environment
 - might be the output of other perceptrons
- Associated with a connection weight $w_d \in \mathbb{R}$, $d = 1, \dots, D$
- Output is some function of the linear combination of inputs
 - $y = s\left(\sum_{j=1}^D w_d x_d + w_0\right) = s(\mathbf{w}^T \mathbf{x})$
where $s(\alpha) = 1$, if $\alpha > 0$, $s(\alpha) = 0$, otherwise
e.g. sigmoid activation: $s(\mathbf{x}, \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$
- can be used for classification, i.e. choose C_1 , if $s(\alpha) > 0.5$

Perceptron: Basic processing unit (2)



- Multiclass: $K > 2$ outputs
 - $y_k = s\left(\sum_{d=1}^D w_{kd}x_d + w_{k0}\right) = s(\mathbf{w}_k^T \mathbf{x})$
where w_{kj} is the weight from input x_j to output y_k
e.g. $s(\mathbf{x}, \mathbf{w}_1, \dots, \mathbf{w}_K) = \frac{\exp(\mathbf{w}_k^T \mathbf{x})}{1 + \sum_{k=1}^K \exp(\mathbf{w}_k^T \mathbf{x})}$
 - 0/1 encoding for output vector
 - e.g. in a 4-class problem: if class=3, then $y = [0, 0, 1, 0]$

Overview (3)

- Perceptron
 - Representation
 - Training
 - Examples
- Multilayer Perceptron
 - Representation
 - Learning: Backpropagation
 - Practical issues

Perceptron: Training (1)

Online training: Stochastic gradient descent

- Evaluation: cross-entropy function for 1 instance $(\mathbf{x}_n, y_n)$
$$\mathcal{E}(\mathbf{w}) = -y_n \log [\sigma(\mathbf{w}^T \mathbf{x}_n)] - (1 - y_n) \log [1 - \sigma(\mathbf{w}^T \mathbf{x}_n)]$$
$$\mathcal{E}(\mathbf{w}_1, \dots, \mathbf{w}_K) = - \sum_{k=1}^K y_{nk} \log p(y_{nk} = 1 | \mathbf{w}_1, \dots, \mathbf{w}_K)$$
- Optimization: gradient descent
$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial w_d} = (\sigma(\mathbf{w}^T \mathbf{x}_n) - y_n) x_{nd}$$
$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial w_{kd}} = (\sigma(\mathbf{w}^T \mathbf{x}_n) - y_{nk}) x_{nd}$$

We could have also performed batch gradient descent.

Perceptron: Training (2)

Online training

- Cost-efficient (computationally and memory-wise)
- Nature of data can change over time
- Error function expressed in terms of individual samples
- Weight update performed after each instance is seen

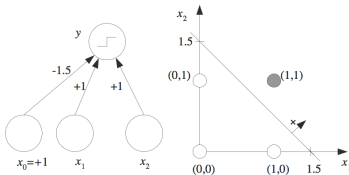
Overview (4)

- Perceptron
 - Representation
 - Training
 - Examples
- Multilayer Perceptron
 - Representation
 - Learning: Backpropagation
 - Practical issues

Approximating linear functions (1)

Example: Boolean AND

x_1	x_2	r
0	0	0
0	1	0
1	0	0
1	1	1



Example of a perceptron implementing AND

$$y = s(x_1 + x_2 - 1.5)$$

$$\mathbf{w} = [-1.5 \ 1 \ 1]^T$$

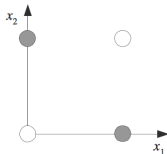
$$\mathbf{x} = [1 \ x_1 \ x_2]^T$$

The above weights were empirically selected, but we could have also learned them through gradient descent

Approximating linear functions (2)

Example: Boolean OR

x_1	x_2	r
0	0	0
0	1	1
1	0	1
1	1	0



Example of a perceptron implementing OR

$$y = s(x_1 + x_2 - 0.5)$$

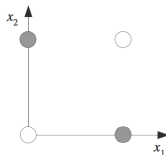
$$\mathbf{w} = [-0.5 \ 1 \ 1]^T$$

$$\mathbf{x} = [1 \ x_1 \ x_2]^T$$

Approximating linear functions (3)

Example: Boolean XOR

x_1	x_2	r
0	0	0
0	1	1
1	0	1
1	1	0



Not linearly separable

Need combination of more than one perceptrons → **multilayer perceptrons**

Multilayer Perceptron: Approximating non-linear functions

Example: Boolean XOR with multilayer perceptrons

x_1	x_2	z_1	z_2	r
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

